

OS- ORANGE PROBLEM 2
NAME: PRATHIKSHA B P
SRN: PES1UG24CS925
SECTION: I

SCREENSHOTS:

1a.

```
prathikshaholla@ubuntunew:~/PES1UG24CS925-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
prathikshaholla@ubuntunew:~/PES1UG24CS925-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aaafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aaafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
prathikshaholla@ubuntunew:~/PES1UG24CS925-pes-vcs$
```

1b.

```
prathikshaholla@ubuntunew:~/PES1UG24CS925-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aaafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
prathikshaholla@ubuntunew:~/PES1UG24CS925-pes-vcs$
```

2a

```
prathikshaholla@ubuntunew:~/PES1UG24CS925-pes-vcs$ make test_tree
make: 'test_tree' is up to date.
prathikshaholla@ubuntunew:~/PES1UG24CS925-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
prathikshaholla@ubuntunew:~/PES1UG24CS925-pes-vcs$
```

2b

```

bash: ./test: file: No such file or directory
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ ./pes init
echo "hello" > hello.txt
echo "world" > main.c
./pes add hello.txt main.c
./pes commit -m "init"
Initialized empty PES repository in .pes/
Committed: f3697bf9c090... init
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ # List all objects with sizes so you can spot the tree
find .pes/objects -type f | xargs ls -la
-rw-r--r-- 1 prathikshaholla prathikshaholla 103 Apr 20 16:14 .pes/objects/2a/c9b634087756e7c325d9ee8818d8e4db23c4125b71042466f1afc42e570209
-rw-r--r-- 1 prathikshaholla prathikshaholla 13 Apr 20 16:14 .pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
-rw-r--r-- 1 prathikshaholla prathikshaholla 13 Apr 20 16:14 .pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
-rw-r--r-- 1 prathikshaholla prathikshaholla 175 Apr 20 16:14 .pes/objects/f3/697bf9c090ba53242b764edd8f6cc1c413683884b1070b84f1f886330b1b98
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ xxd .pes/objects/XX/YYYY... | head -20
xxd: .pes/objects/XX/YYYY...: No such file or directory
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ # This prints all objects with their type
for f in $(find .pes/objects -type f); do
  hash=$(echo $f | sed 's|.pes/objects/||' | tr -d '/')
  echo "$hash: $(head -c 20 $f | strings | head -1)"
done
2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4: blob 6
f3697bf9c090ba53242b764edd8f6cc1c413683884b1070b84f1f886330b1b98: commit 164
2ac9b634087756e7c325d9ee8818d8e4db23c4125b71042466f1afc42e570209: tree 95
e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f: blob 6
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ xxd .pes/objects/2a/c9b634087756e7c325d9ee8818d8e4db23c4125b71042466f1afc42e570209 | head
-20
00000000: 7472 6565 2039 3500 3130 3036 3434 2068   tree 95.100644 h
00000010: 656c 6c6f 2e74 7874 002c f0d8 3d9e e295   ello.txt,...=...
00000020: 43b3 4a87 7274 21fd ecb7 e3f3 a183 d337   C.J.rtl!.....7
00000030: 6390 25de 576d b9eb b431 3030 3634 3420   c.%.Wm...100644
00000040: 6d61 696e 2e63 00e0 0c50 e16a 2df3 8f0d   main.c...P.j-...
00000050: 6bf8 09e1 81ad 0248 da6e 6719 f35f 9f7e   k.....H.ng.~.~
00000060: 65d6 f006 199f 7f                                e.....
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$

```

3a

```
00000000: 0500 1000 199f 7f .....
prathikshaholla@ubuntu:~/Documents/PES1UG24CS925-pes-vcs$ # Clean slate
make clean
make all
rm -rf .pes
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
/usr/bin/ld: tree.o: in function 'tree_from_index':
tree.c:(.text+0x743): undefined reference to 'index_load'
collect2: error: ld returned 1 exit status
make: *** [Makefile:22: test_tree] Error 1
prathikshaholla@ubuntu:~/Documents/PES1UG24CS925-pes-vcs$ # Init the repo
./pes init
Initialized empty PES repository in .pes/
prathikshaholla@ubuntu:~/Documents/PES1UG24CS925-pes-vcs$ # Create test files
echo "hello" > file1.txt
echo "world" > file2.txt
prathikshaholla@ubuntu:~/Documents/PES1UG24CS925-pes-vcs$ # Stage them
./pes add file1.txt file2.txt
prathikshaholla@ubuntu:~/Documents/PES1UG24CS925-pes-vcs$ # Check status
./pes status
Staged changes:
  staged:    file1.txt
  staged:    file2.txt

Unstaged changes:
(nothing to show)

Untracked files:
  untracked: test_objects.c
  untracked: .gitignore
  untracked: hello.txt
  untracked: commit.c
  untracked: commit.h
  untracked: test_sequence.sh
  untracked: index.c
  untracked: main.c
  untracked: tree.h
  untracked: tree.c
  untracked: test_tree.c
  untracked: test_objects
  untracked: pes.c
  untracked: pes.h
  untracked: index.h
  untracked: Makefile
  untracked: object.c
  untracked: README.md
prathikshaholla@ubuntu:~/Documents/PES1UG24CS925-pes-vcs$
```

3b

```
cat: .pes/index: No such file or directory
prathikshaholla@ubuntu:~/Documents/PES1UG24CS925-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776702024 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776702024 6 file2.txt
prathikshaholla@ubuntu:~/Documents/PES1UG24CS925-pes-vcs$
```

4a

```
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ # Clean slate again
rm -rf .pes
./pes init
Initialized empty PES repository in .pes/
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ # First commit
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"
Committed: 1432da8d4da3... Initial commit
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ # Second commit
echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
Committed: 6e5fde11d865... Add world
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ # Third commit
echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
Committed: 6aa20e56abbd... Add farewell
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ # View the commit log
./pes log
commit 6aa20e56abbd613faa9f08665edce906b6b1faed2311fc7d77cca1f16392d7b2
Author: PES User <pes@localhost>
Date: 1776702301

    Add farewell

commit 6e5fde11d865286acbccbbe161d34be47d2851e38ed373a04765c927fb1a9b4a
Author: PES User <pes@localhost>
Date: 1776702292

    Add worldell

commit 1432da8d4da3e53bb3715e1e82f5b204fe38008bae17882fe5ad2b6b4a325fe5
Author: PES User <pes@localhost>
Date: 1776702279

    Initial committial commiQ

prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$
```

4b

```
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ # View all objects created
find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/14/32da8d4da3e53bb3715e1e82f5b204fe38008bae17882fe5ad2b6b4a325fe5
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6a/a20e56abbd613faa9f08665edce906b6b1faed2311fc7d77cca1f16392d7b2
.pes/objects/6e/5fde11d865286acbccbbe161d34be47d2851e38ed373a04765c927fb1a9b4a
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/main
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$
```

4c

```
=== All integration tests completed ===
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ # View the reference chain
cat .pes/refs/heads/main
cat .pes/HEAD
6aa20e56abbd613faa9f08665edce906b6b1faed2311fc7d77cca1f16392d7b2
ref: refs/heads/main
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$
```

Final:

```
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$ # Run the full integration test
make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:    file.txt
  staged:    hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)

--- First Commit ---
Committed: 24a111c97435... Initial commit

Log after first commit:
commit 24a111c97435c7fa8a7d1f4381055689b0bfdfbf6036dff702282aabf392e72e
Author: PES User <pes@localhost>
Date:   1776702381

    Initial commit

--- Second Commit ---
Committed: 9ae81536204a... Update file.txt

--- Third Commit ---
Committed: b27a33a9c8d4... Add farewell

--- Full History ---
commit b27a33a9c8d4fe6128f8b97e9b3e2530fae58e0640432c8a743242cf538685d2
Author: PES User <pes@localhost>
```

```
Committed: b27a33a9c8d4... Add farewell
```

```
--- Full History ---
```

```
commit b27a33a9c8d4fe6128f8b97e9b3e2530fae58e0640432c8a743242cf538685d2
```

```
Author: PES User <pes@localhost>
```

```
Date: 1776702381
```

```
    Add farewell
```

```
commit 9ae81536204ad80846c122923ea8241ec813a1b81206adc610deb43aae82b5d1
```

```
Author: PES User <pes@localhost>
```

```
Date: 1776702381
```

```
    Update file.txt
```

```
commit 24a111c97435c7fa8a7d1f4381055689b0bfd6f6036dff702282aabf392e72e
```

```
Author: PES User <pes@localhost>
```

```
Date: 1776702381
```

```
Initial committial commiQ
```

```
--- Reference Chain ---
```

```
HEAD:
```

```
ref: refs/heads/main
```

```
refs/heads/main:
```

```
b27a33a9c8d4fe6128f8b97e9b3e2530fae58e0640432c8a743242cf538685d2
```

```
--- Object Store ---
```

```
Objects created:
```

```
10
```

```
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
```

```
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
```

```
.pes/objects/24/a111c97435c7fa8a7d1f4381055689b0bfd6f6036dff702282aabf392e72e
```

```
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
```

```
.pes/objects/9a/e81536204ad80846c122923ea8241ec813a1b81206adc610deb43aae82b5d1
```

```
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
```

```
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
```

```
.pes/objects/b2/7a33a9c8d4fe6128f8b97e9b3e2530fae58e0640432c8a743242cf538685d2
```

```
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48baf6aa3829f61f75b8d10d5350f9
```

```
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
```

```
=== All integration tests completed ===
```

```
prathikshaholla@ubuntunew:~/Documents/PES1UG24CS925-pes-vcs$
```

Analysis Answers

Q5.1: How `pes checkout <branch>` should work

1. Resolve the target branch ref from `.pes/refs/heads/<branch>` and read the target commit hash.
2. Read target commit object, then its root tree.
3. Update `.pes/HEAD` to `ref: refs/heads/<branch>` (or keep detached mode logic separately).
4. Rewrite working directory to match target tree snapshot:
 - Create missing files/directories.
 - Overwrite tracked files that differ.
 - Remove tracked files/directories that are not present in target tree.
5. Rewrite `.pes/index` so it matches the checked-out tree state.

Complexity comes from safe working-tree updates, conflict detection with local modifications, directory recursion, file deletions, and atomicity guarantees if checkout fails midway.

Q5.2: Dirty working directory conflict detection using index + object store

For each tracked path in index:

1. Compare current filesystem metadata (mtime/size) with index entry.
2. If metadata differs, hash working file content and compare with index blob hash.
3. If hash differs, file is dirty.

Then compare the current commit tree path hash vs target branch tree path hash:

1. If file is dirty in working directory.
2. And file content differs between current tree and target tree.
3. Refuse checkout for that path because switching would overwrite uncommitted work.

This is equivalent to checking three versions: working tree, current snapshot, target snapshot.

Q5.3: Detached HEAD behavior and recovery

In detached HEAD, new commits are created normally, but no branch name moves with them. Only HEAD points to the latest commit hash. Once user switches away, those commits can become unreachable from branch refs.

Recovery options:

1. Immediately create a branch pointing to current detached commit (`refs/heads/<new-branch>`).

2. If already moved away, recover via reflog-like history (if implemented) or from remembered commit hash.
3. After creating a branch ref, commits are reachable again and protected from GC.

Q6.1: Unreachable object GC algorithm

Mark-and-sweep approach:

1. Initialize a hash-set `reachable`.
2. Seed traversal from all refs in `.pes/refs/heads/*` and optionally tags.
3. DFS/BFS commits:
 - Mark commit object.
 - Visit parent commit(s).
 - Visit commit's tree.
4. DFS/BFS trees:
 - Mark tree object.
 - For each entry, mark referenced blob/tree and recurse for subtrees.
5. Enumerate all files in `.pes/objects/**`.
6. Delete any object not in `reachable`.

Efficient structure: hash set keyed by 32-byte binary hash (or hex string).

For 100,000 commits and 50 branches, visited commit count is typically near unique commit count (about 100,000, not 5,000,000) because branch histories overlap heavily. Total objects visited is commits plus reachable trees/blobs, usually several times commit count depending on project churn.

Q6.2: Why concurrent GC is dangerous

Race example:

1. Commit process writes new blob/tree objects.
2. Before branch ref is updated to the new commit, those objects are temporarily unreachable from refs.
3. Concurrent GC scans refs, does not see the new commit path, and deletes those newly written objects.
4. Commit process then updates ref to point at a commit referencing missing objects, corrupting repository history.

How Git avoids this:

1. Uses object reachability grace periods and reflogs.
2. Uses packfile and lock protocols to coordinate writers/GC.
3. Avoids pruning very recent unreachable objects (`gc.pruneExpire` default window).
4. Uses atomic ref updates and filesystem locks to reduce races.